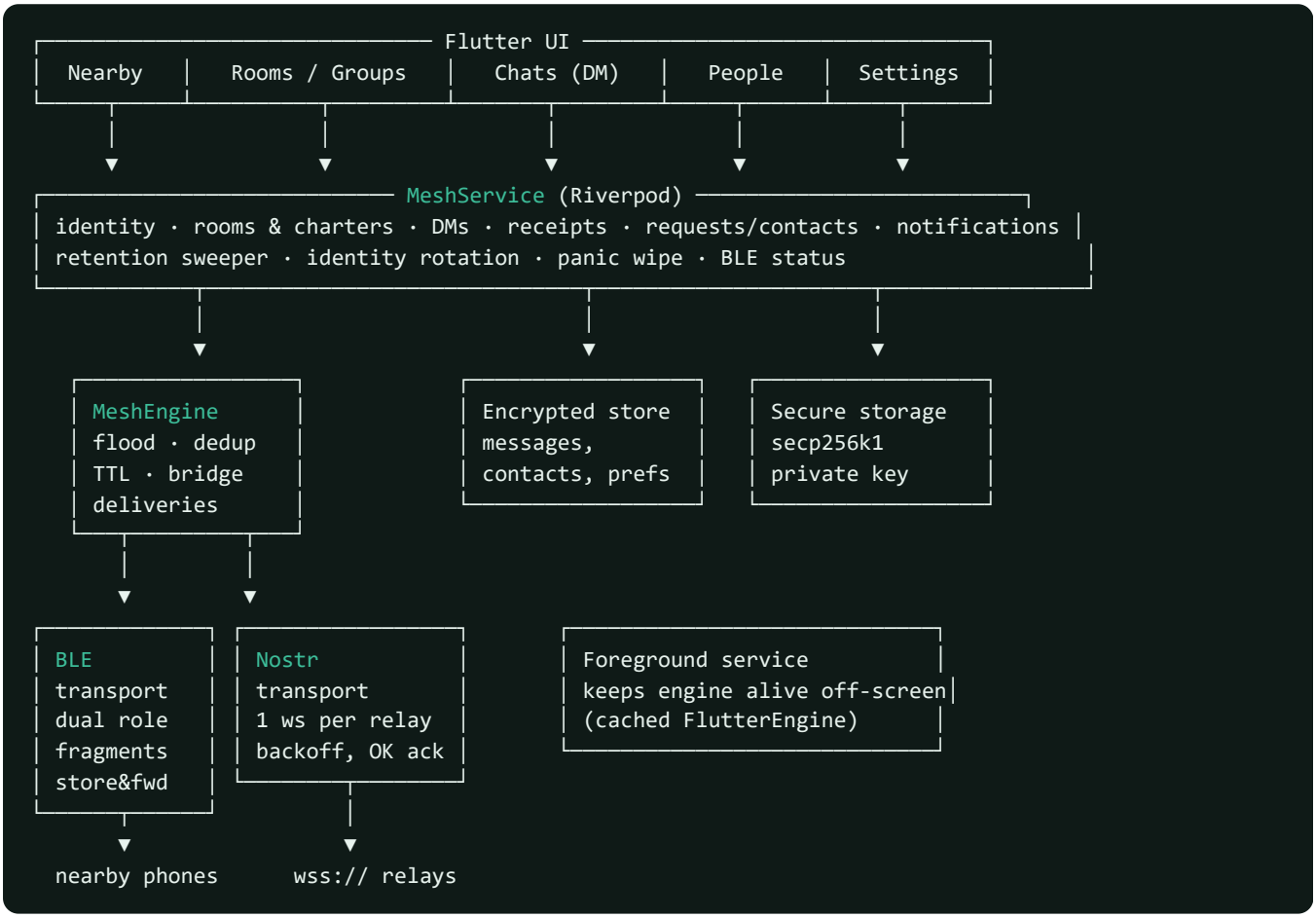
Architecture

Components, data flow, wire format, cryptography, storage, threat model.

Document 4 of 4 · Version 1.0 · For engineers and reviewers · Contact: keshavgithub01@gmail.com

1. Components



The engine is transport-agnostic: it receives `Inbound(frame, via, peerId)` from either transport and emits frames to both. The service layer owns all semantics (rooms, DMs, governance). The UI is a thin Riverpod consumer.

Source layout

```
lib/core/crypto      keys (BIP-340 Schnorr), NIP-44, Argon2id, XChaCha20-Poly1305, bytes
lib/core/identity    Identity, readable names, avatar/colour derivation, nip19 (npub)
lib/core/models      Frame, Message, Room, Charter, DeliveryLevel …
lib/core/storage     encrypted store, settings (SharedPreferences), secure key storage
lib/transport        mesh_engine, store_forward, ble/ble_transport, nostr/{nostr_transport,relay_c
onnection,nostr_event}
lib/services         mesh_service, notifications, foreground_service, native (method channel)
```

```
lib/features         chat, rooms, dm, people, settings (screens + Riverpod state)
lib/ui               app (router, theme), shell (tabs), logo, avatar, bluetooth_card, setup_sheet,
in_app_alert
android/…/MainActivity.kt, LibRateApplication.kt   FLAG_SECURE, apkPath, BT enable, openUrl, cache
d engine
tool/                relay test tools (pub, sub, who, dm)
```

2. Wire format

A frame is a compact binary structure. Fields in order:

version	1 byte
type	1 byte — room text, DM, charter, ban, dismantle, hand-off, ack, cover
ttl	1 byte — starts at 7, decremented per hop, **zeroed when computing the signature**
timestamp	8 bytes, ms since epoch
sender	32-byte x-only public key
target	room tag (UTF-8, length-prefixed) or recipient pubkey for DMs
payload	length-prefixed bytes — plaintext, or ciphertext for DMs / password rooms
signature	64-byte BIP-340 Schnorr over all preceding bytes with ttl = 0

Frame id = SHA-256 of the signed bytes (ttl zeroed). Identical on Bluetooth and relay paths. Frames larger than a BLE link's MTU are split into numbered fragments per link and reassembled by the peer; fragmentation is a link-layer concern and never changes the frame or its id.

Nostr mapping

Each frame is carried as a Nostr event: `kind 23333` for room traffic and `kind 24444` for direct traffic (both ephemeral-range, so relays need not persist them). The event `content` is the base64 frame; tags carry the room tag for subscription filtering. The event is signed with the same key, so relays enforce authenticity too. A relay's `OK` reply advances the tick to *sent*.

3. Routing (MeshEngine)

```
inbound(frame, via)
  ├─ verify signature ──────────────── fail → drop
  ├─ seen.contains(id)? ─────────────── yes  → drop            (dedup set, time-bounded)
  ├─ seen.add(id)
  ├─ deliver → MeshService              (room joined / DM for me / governance record)
  ├─ ttl > 1 ?  → ttl-1 → BLE.sendAll(except origin link)   (flood once)
  ├─ via == BLE  && relays connected → Nostr.publish        (bridge up)
  └─ via == Nostr                    → BLE.sendAll            (bridge down)

outbound(frame)
  ├─ seen.add(id); deliver locally as pending
```

```
   ├─ BLE.sendAll        → first peer accept → delivered (on echo)
   └─ Nostr.publish      → OK ack → sent; no relay → queued, retried on connect
```

Per-conversation delivery state flows back through a `deliveries` stream. Store-and-forward keeps recent frames keyed by target; when a new BLE link appears, frames addressed to that peer (or, before it is identified, all held frames) are replayed.

4. Bluetooth transport

- **Dual role.** Each phone advertises a GATT service (peripheral) *and* scans for it (central). Two independent state machines; each restarts on its own with backoff.
- **GATT.** One service UUID; `RX` characteristic (central writes to peripheral) and `TX` (peripheral notifies central). Both directions fragment to the negotiated MTU (request 517; fall back to 180-byte chunks when the stack reports the 20-byte default).
- **Duty cycle.** Scanning is windowed to save power; *battery saver* shortens the window (2 s every 22 s). Links are capped per role.
- **Transient failures** (e.g. GATT status 8, supervision timeout) are treated as noise: the link is cleaned up, the peer is rediscovered, and the UI shows a hint rather than an error once relinked.
- **Permissions.** `BLUETOOTH_SCAN / CONNECT / ADVERTISE` (Android 12+) requested from the UI with an Activity; the foreground service never prompts.

5. Cryptography

Identity	secp256k1; x-only public key; BIP-340 Schnorr signatures; `npub` bech32 encoding.
Direct messages	NIP-44 v2: ECDH → HKDF-SHA256 conversation key → per-message nonce → ChaCha20 + HMAC-SHA256, padded. Verified against the published test vectors. Read receipts are NIP-44 payloads `{"ack":[ids]}`.
Password rooms	Argon2id(password, salt = room tag) → 32-byte key; XChaCha20-Poly1305 with a random 24-byte nonce per message. Tag is derived from the name, so the password never appears on the wire.
Governed groups	Tag `#name.<16 hex of creator key>`. Charter = signed(admins, description). Ban = signed(member, seq) by an admin — higher seq wins. Dismantle = signed by creator. Each phone validates against the creator key embedded in the tag.
Identity rotation	Hand-off record signed by the *old* key naming the new key; recipients re-key the contact locally. Groups created by the old key are dismantled.
At rest	Message store encrypted with a device key held in Android Keystore-backed secure storage; the private key lives in secure storage only. `allowBackup=false`.

6. Storage and lifecycle

- **Retention**: `0` forever · `N` days (sweeper) · `-1` memory only (never written).
- **Settings** in SharedPreferences: relays, joined rooms, retention, cover traffic, battery saver, screenshots, onboarding, setup-hint snooze, notifications, theme.
- **Background**: an Application-owned cached `FlutterEngine` survives Activity destruction; a foreground service with a persistent notification keeps the process alive. Battery-optimisation exemption is requested once and re-checked on resume.
- **Notifications**: channels `librate_dm` / `librate_rooms`, visibility *private*, per-conversation mutes; in-app banner overlay when the app is open on another screen; Nearby never notifies.
- **Navigation**: Nearby · Rooms · Chats · Settings tabs; conversations are children of their tab; tapping a tab always returns to its list.

7. Build and release

```
flutter pub get && flutter analyze
flutter build apk --release --split-per-abi --obfuscate --split-debug-info=symbols
```

- Flutter 3.47 / Dart 3.13 · `minSdk 26`, `targetSdk 36` · package `com.librate.mesh`.
- Release signing from `android/key.properties` (git-ignored) → keystore in `android/keys/` (git-ignored). Certificate SHA-256 `7F:05:8A:F1:48:8C:A4:0A:AF:B2:14:86:D7:7A:64:24:19:9B:00:30:BB:38:A0:44:FA:11:B7:8D:7A:78:15:B6`.
- Per-ABI APKs with `SHA256SUMS.txt`; obfuscation symbols kept offline for crash symbolication.
- Resource shrinker keeps the notification icon via `res/raw/keep.xml`; core-library desugaring enabled.

8. Threat model

Adversary	Can	Cannot
Bystander with a Bluetooth sniffer	See that LibRate is running; read Nearby and open-room text; observe traffic timing.	Read DMs or password rooms; forge messages; identify the person behind a key.
Relay operator / network observer	See public-room frames, sender keys, timestamps, client IPs.	Read DMs, password rooms or receipts; alter frames; tell which phone originated a bridged frame.
Someone with the APK	Build a modified app.	Install it over a genuine one (different signature); make it pass a hash check.
Someone holding an unlocked phone	Read everything on it.	— Mitigations: screenshots blocked, panic wipe, memory-only retention, no backup.
A chat partner	Screenshot or forward what you wrote.	Prove you wrote it to a third party without your key? They can — signatures are non-repudiable. Say less.

Known limitations. DM recipient keys are visible on the wire (metadata). Message IDs and timestamps allow traffic analysis. Hop counts leak rough topology. Cover traffic reduces, not removes, timing analysis. No forward secrecy in NIP-44 — a leaked private key decrypts past DMs that are still stored.

9. Extending

- **New transport** (e.g. Wi-Fi Aware): implement the transport interface (inbound stream, `send`, `peers`) and register it with the engine — no change to frames or services.
- **New relay**: any Nostr relay; see `docs/RELAY.md` for strfry + Caddy and mirroring.
- **Probabilistic rebroadcast** for very dense crowds and a configurable TTL are straightforward engine changes if field data shows air-time saturation.